

Queue Cure 2026 - Hackathon Supporting Documentation

1. Socket Event Diagram

Real-Time Communication Flow



Socket Events

Event	Trigger	Purpose
-------	---------	---------

patient_added	New patient added	Update all screens
patient_removed	Patient removed	Refresh queue
patient_called	Next token called	Update current token
queue_updated	Queue state changes	Synchronize clients
settings_updated	Consultation time changed	Recalculate wait times
queue_reset	Queue cleared	Reset all screens
analytics_updated	Analytics recalculated	Refresh metrics
request_state	Client reconnects	Recover latest state

Example Flow: Patient Added



Why WebSockets?

Traditional polling:

Client → Request → Server → Response → Repeat

Problems:

- Delayed updates
- Higher server load
- Poor user experience

Queue Cure 2026 WebSockets:

Server ↔ Client

Benefits:

- Instant updates
- Lower bandwidth
- Better scalability
- Improved patient experience

2. Thought Process Document

Problem Statement

Many clinics still rely on paper-token systems, manual announcements, and disconnected communication between receptionists and patients. This leads to:

- Patients unaware of their position or wait time
- Receptionists repeating status updates verbally
- No visibility into daily throughput or analytics
- Disconnected experience between staff and waiting patients

Why This Problem Matters

- Paper tokens are inefficient and error-prone
- Manual announcements cause frustration and confusion
- Lack of real-time data hurts clinic operations
- Poor patient experience affects satisfaction

Solution Overview

Queue Cure 2026 is a modern real-time healthcare queue management system that:

- Provides live queue visibility
- Eliminates manual announcements
- Calculates wait times automatically
- Tracks doctor status
- Generates operational analytics
- Uses WebSockets for instant updates

Architecture Decisions

Component	Choice	Reason
Backend	Flask	Lightweight, easy to deploy, great for hackathons
Real-Time	Flask-SocketIO	Native WebSocket support for instant updates
Database	SQLite	File-based, no server needed, perfect for small clinics
Frontend	Bootstrap 5	Responsive, professional, fast to build
Deployment	Render	Free tier, auto-config from render.yaml, persistent disk

Why Flask-SocketIO Was Chosen

- Seamless integration with Flask
- Automatic fallback mechanisms
- Easy to broadcast events to multiple clients
- Works great with gunicorn + eventlet
- Perfect for small to medium clinic deployments

Real-Time Communication Strategy

- Every user action on the dashboard triggers a Socket.IO event
- Server updates the database then broadcasts the event
- All connected clients (dashboard, waiting room, analytics) receive the update
- No polling, no manual refresh

Concurrency Handling

- SQLite serializes writes, preventing race conditions
- Flask-SocketIO + eventlet worker supports multiple concurrent connections
- Multi-tab updates work perfectly via broadcast events

Edge Cases Handled

- Empty queue: Call Next shows error, no patient displayed
- Duplicate patient names: 409 Conflict response, patient not added
- Invalid consultation time: Validates between 1-120 minutes
- Multiple tabs: All tabs update simultaneously via Socket.IO
- Browser reconnect: Client sends `request_state` event to recover latest data
- Simultaneous updates: Server handles in order via SQLite serialization

Scalability Considerations

- Current setup perfect for small/medium clinics
- For larger clinics:
- Switch to PostgreSQL
- Use Redis for Socket.IO message broker
- Add load balancing
- Implement user authentication

Future Improvements

- SMS/WhatsApp notifications when patient's turn is approaching
- Multi-doctor/multi-room support
- Appointment scheduling integration
- QR code check-in for patients
- Historical analytics with date range filters

- PWA support for mobile receptionist app
- Role-based access control

3. API Documentation

Patients API

Method	Endpoint	Purpose	Request Body	Response Example
GET	/api/patients	Get all patients (active queue + history)	N/A	<pre>{ "patients": [{ "id": 1, "token_number": "QC-123456", "patient_name": "Alice", "status": "waiting", ... }] }</pre>
POST	/api/patients	Add new patient	<pre>{ "patient_name": "Alice", ... }</pre>	<pre>{ "success": true, "patient": { ... } }</pre>
POST	/api/patients/call-next	Call next patient in queue	N/A	<pre>{ "success": true, "patient": { ... } }</pre>
POST	/api/patients/:id/call	Call specific patient	N/A	<pre>{ "success": true, "patient": { ... } }</pre>
DELETE	/api/patients/:id	Remove patient from queue	N/A	<pre>{ "success": true }</pre>
POST	/api/patients/:id/complete	Mark patient's consultation as complete	N/A	<pre>{ "success": true, "patient": { ... } }</pre>

Settings API

Method	Endpoint	Purpose	Request Body	Response Example
GET	/api/settings	Get clinic settings	N/A	<pre>{"average_consultation_time":15, "doctor_status":"available", "clinic_name":"Queen's Cure Clinic"}</pre>
PUT	/api/settings	Update clinic settings	<pre>{"average_consultation_time":15, "doctor_status":{"success":true, "clinic_name":"settings":{"...}} Clinic"}</pre>	

Queue API

Method	Endpoint	Purpose	Request Body	Response Example
POST	/api/queue/reset	Reset active queue (keep history)	N/A	<pre>{"success":true}</pre>

Analytics API

Method	Endpoint	Purpose	Request Body	Response Example
GET	/api/analytics	Get analytics metrics	N/A	<pre>{"served_today":5, "waiting_count":3, "avg_wait_time":15, ...}</pre>

GET	/api/export	Export patient history as CSV	N/A	CSV file download
-----	-------------	-------------------------------	-----	-------------------

4. Test Cases

Functional Tests

Test Case	Steps	Expected Result	Actual Result
Add Patient	- Go to Dashboard - Enter patient name - Click "Add"	Patient added, token assigned, all screens update	■ Passing
Remove Patient	- Add patient - Click "Remove" - Confirm	Patient removed, queue updates	■ Passing
Call Next Token	- Add patient - Click "Call Next"	Patient marked as called, current token updates	■ Passing
Queue Reset	- Add patients - Click "Reset Queue" - Confirm	Queue cleared, screens reset	■ Passing
Change Consultation Time	- Go to Settings - Enter new time (1-120) - Save	Time saved, wait times recalculated	■ Passing

Edge Cases

Test Case	Steps	Expected Result	Actual Result
Empty Queue Call Next	1. Reset queue 2. Click "Call Next"	Error message: "Queue is empty"	■ Passing
Duplicate Patient Names	1. Add "Alice" 2. Try adding "Alice" again	409 Conflict: Patient already in queue	■ Passing
Invalid Consultation Time	1. Go to Settings 2. Enter 0 or 121 3. Save	Validation error	■ Passing
Multiple Tabs Open	1. Open 2 Dashboard tabs 2. Add patient in one tab	Second tab updates instantly	■ Passing
Browser Reconnect	1. Open Waiting Room 2. Refresh page	Page connects and recovers latest state	■ Passing
Simultaneous Updates	1. Open 2 Dashboard tabs 2. Add patient from both at same time	Both tabs update, no duplicate tokens	■ Passing

5. Deployment Guide

Local Setup

Clone repo:

```
bash git clone https://github.com/VanitaCSE/queue-cure-2026.git cd queue-cure-2026/queue-cure
```

Create virtual environment:

```
``bash
```

```
# Windows
```

```
python -m venv venv
```

```
.\venv\Scripts\activate
```

```
# macOS/Linux
```

```
python3 -m venv venv
```

```
source venv/bin/activate
```

```
``
```

Install requirements:

```
bash pip install -r requirements.txt
```

Run Flask app:

```
bash python app.py
```

Open in browser:

4. Landing: <http://localhost:5000>
5. Dashboard: <http://localhost:5000/dashboard>
6. Waiting Room: <http://localhost:5000/patient>
7. Analytics: <http://localhost:5000/analytics>

GitHub Push Process

1. Make changes locally
2. Stage, commit, push:

```
bash git add . git commit -m "Your commit message" git push origin  
main
```

Render Deployment Process

1. Log in to <https://render.com>
2. Click **"New"** → **"Web Service"**
3. Select your GitHub repo: [VanitaCSE/queue-cure-2026](#)
4. Configure (render.yaml auto-fills!):
5. Name: [queue-cure-2026](#)
6. Runtime: Python 3

7. Branch: main
8. Build Command: `pip install -r requirements.txt`
9. Start Command: `gunicorn --worker-class eventlet -w 1 --bind 0.0.0.0:$PORT app:app`
10. Plan: Free
11. Click **"Create Web Service"**

Environment Variables

Key	Value
PYTHON_VERSION	3.11.9
SECRET_KEY	Auto-generated by Render
DATABASE_PATH	/var/data/database.db

Verification Checklist

- ☐ App loads without errors
- ☐ Dashboard accessible
- ☐ Patient can be added
- ☐ Real-time updates work
- ☐ Waiting Room updates
- ☐ Analytics dashboard loads
- ☐ CSV export works
- ☐ Dark mode toggle works
- ☐ Doctor status updates
- ☐ All tests pass locally
- ☐ No console errors

6. Verification and Testing

Run Tests Locally

```
# Install dev requirements
pip install -r requirements-dev.txt
|
# Run all tests
pytest tests/ -v
```

Screenshots

See the Screenshots directory in the repo for images of all pages!

Copyright and License

MIT License - Built for hackathon submission, Queue Cure 2026.